

Computer Organisation

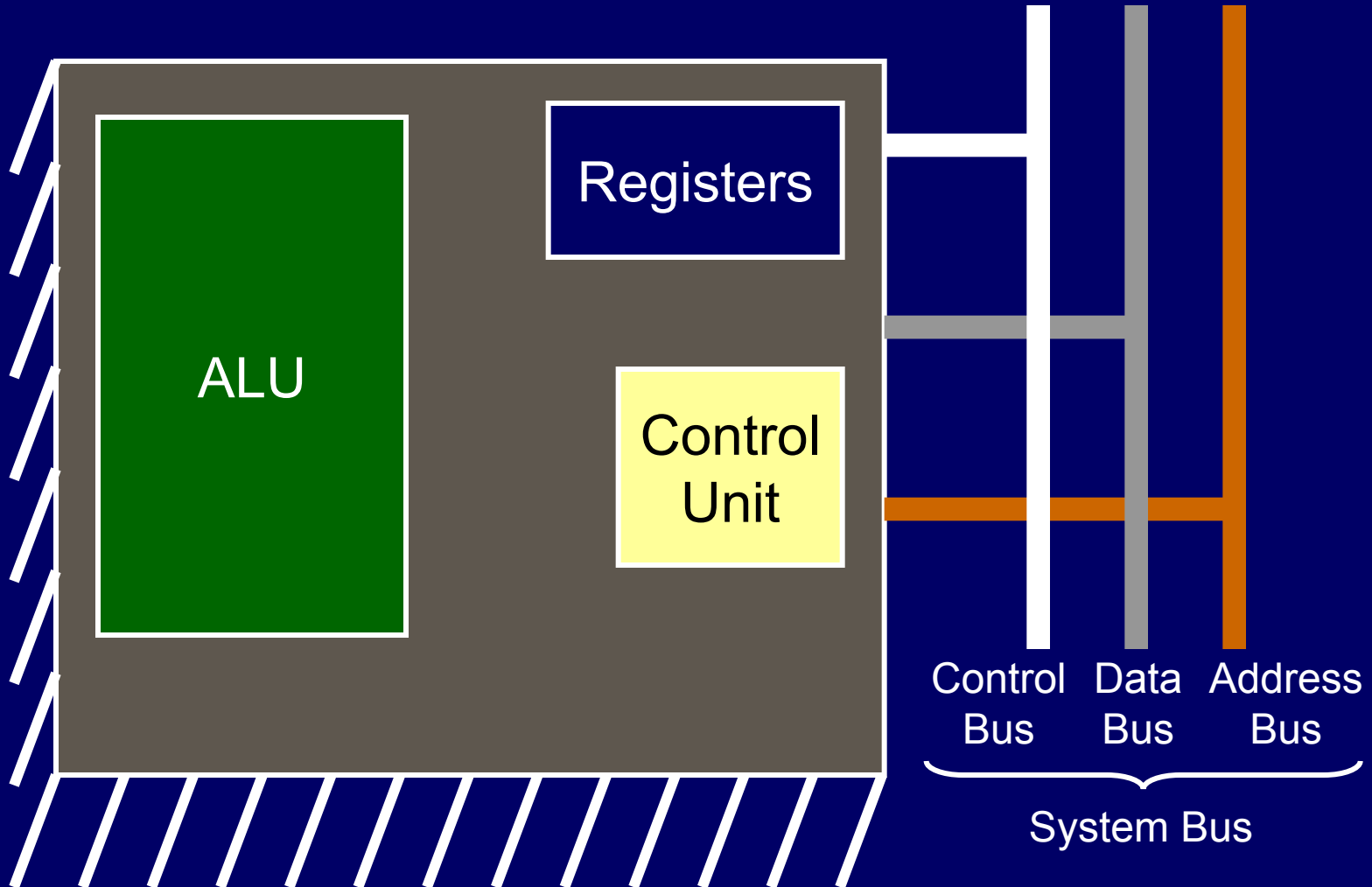
Topic 7 Central Processing Unit

CPU Structure (1)

- CPU must:
 - Fetch instructions (from memory)
 - Interpret instructions (decode)
 - Fetch data (from memory or IO module)
 - Process data (arithmetic or logic)
 - Write data (to memory or IO module)
- CPU components:
 - Arithmetic and Logic Unit (ALU) - does computation
 - Control Unit (CU) - controls data/instruction movement
 - Registers - internal memory

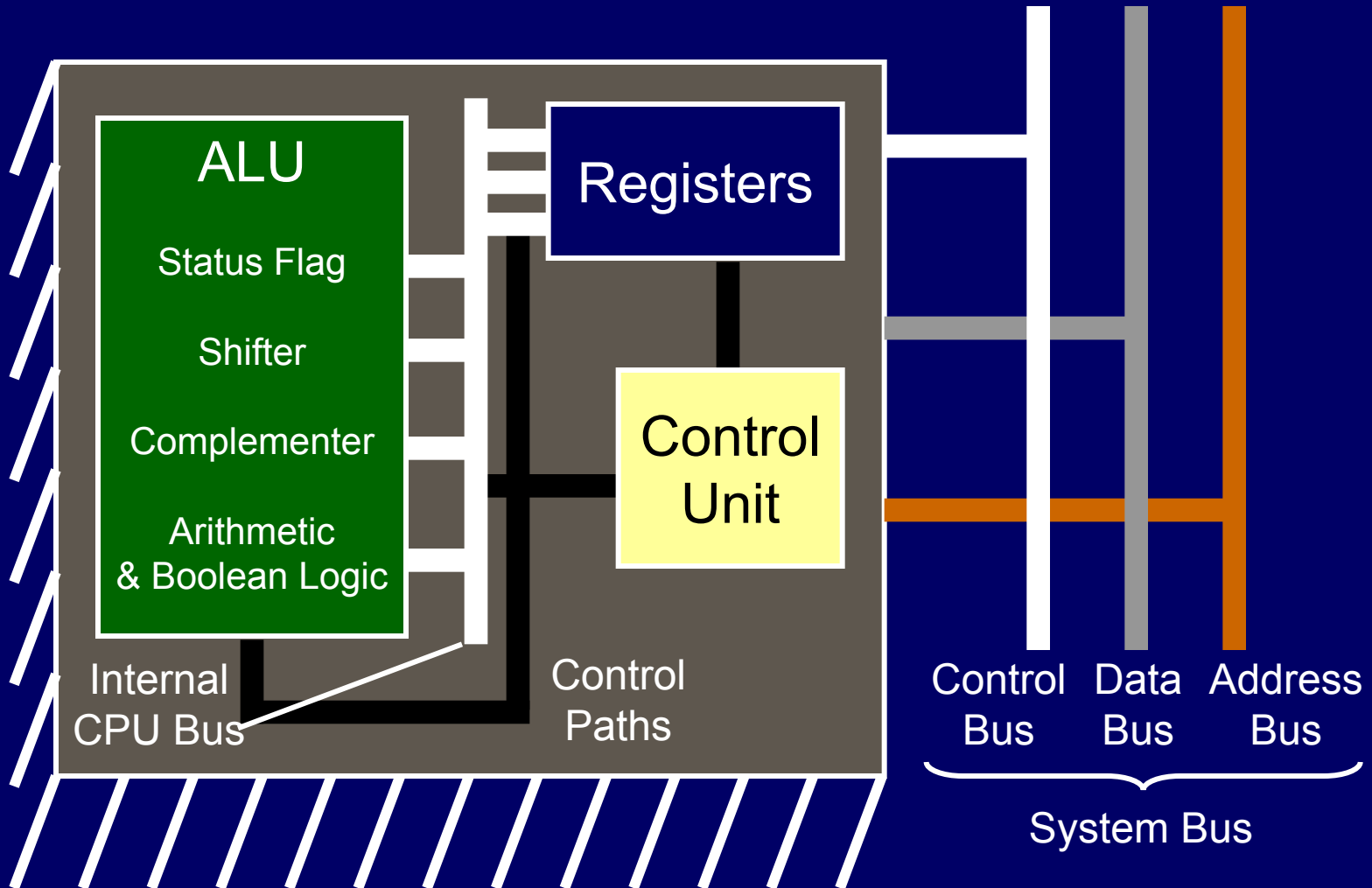
CPU Structure (2)

CPU with Systems Bus



CPU Structure (3)

Internal Structure of CPU



Registers

- CPU must have some working space (temporary storage) - called Registers
- Number and function vary between processor designs
- One of the major design decisions
- Top level of memory hierarchy (above cache and main memory)
- 2 functions:
 - User-visible registers
 - Control and status registers

Instruction Cycle

- Instruction cycle includes the following subcycles:
 - Fetch - read next instruction from memory into CPU
 - Execute - interpret opcode and perform operation
 - Interrupt - save current process state and service interrupt
 - Indirect – more memory access to fetch operand

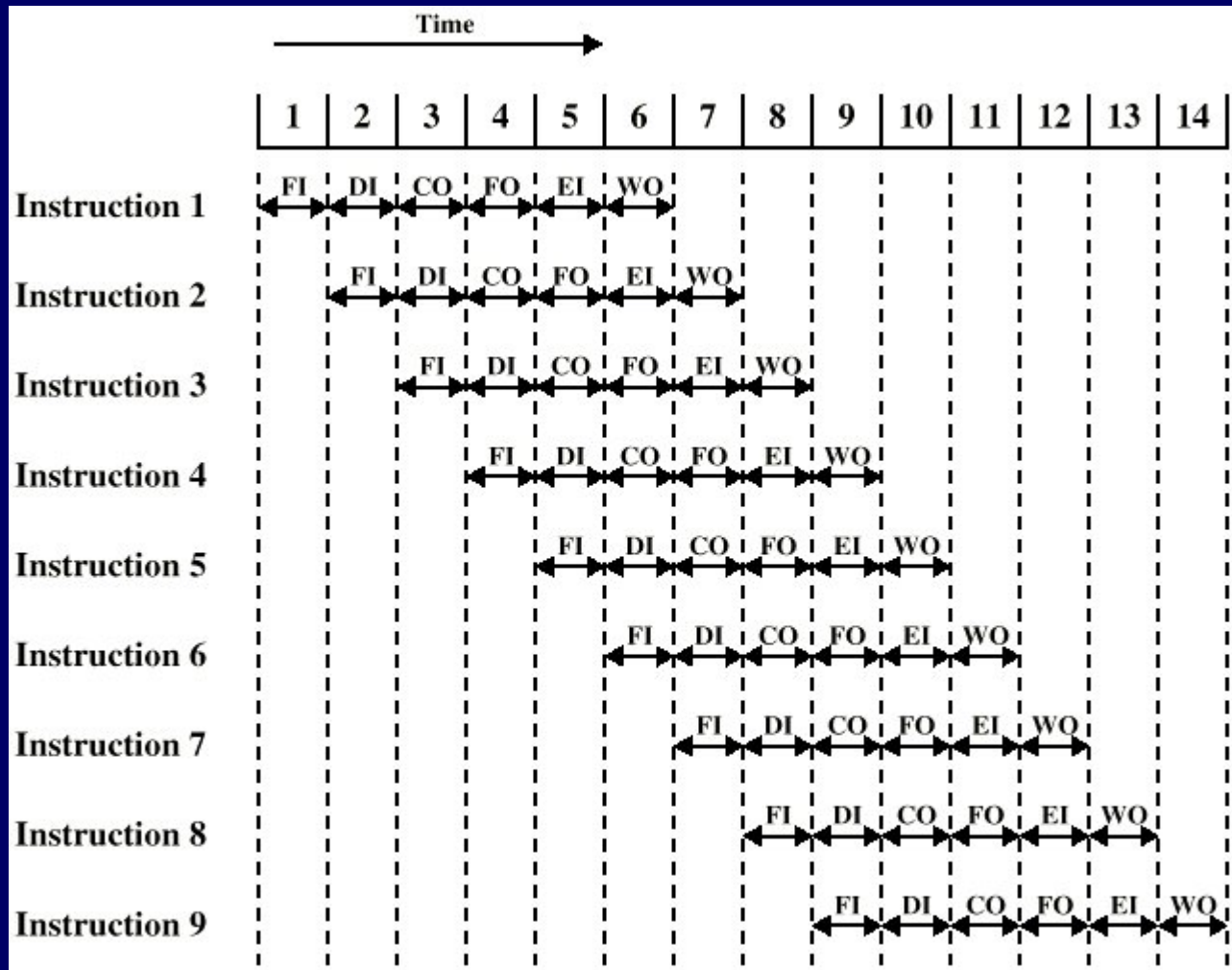
Instruction Pipelining

Prefetch

- Analogy: assembly line
 - Various stages of product can be worked on simultaneously = pipelining
- For computers:
 - Execution usually does not access main memory
 - Fetch accessing main memory
- Can fetch next instruction during execution of current instruction (simple 2-stage pipeline)
- Called instruction prefetch



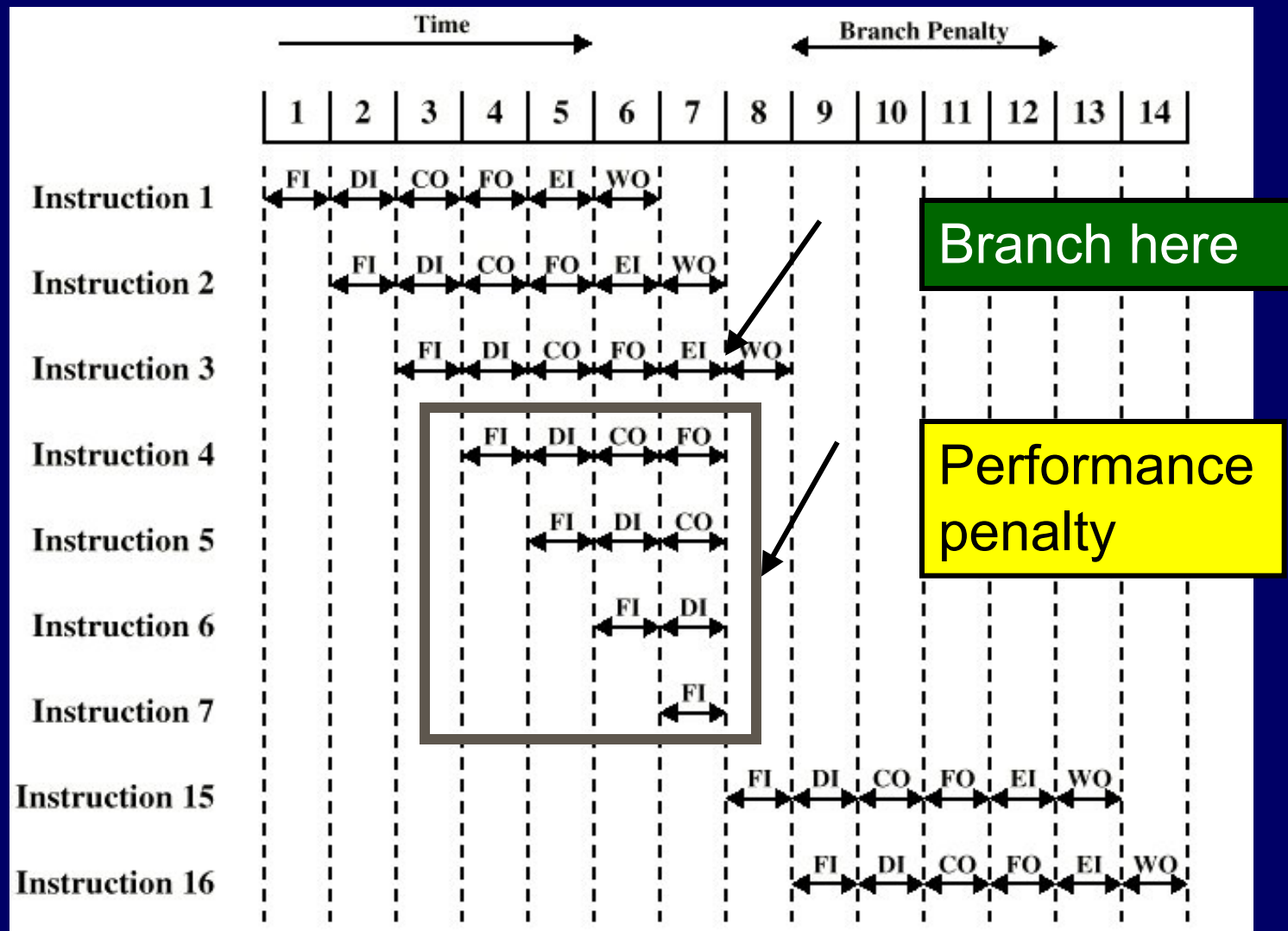
Timing of Pipeline (1)



Pipelining Performance Limits

- Waiting involved if stages are not of equal duration (like in 2-stage pipeline)
- Conditional branches
- Interrupts
- Calculate Operand stage may see conflict in memory and register
- Increasing number of stages not necessary improve performance:
 - Overhead of moving data from buffer to buffer & performing preparatory and delivery functions
 - More control logic required- more complex than stages

Branch in a Pipeline



Pipeline Performance (1)

- Without pipeline,

$$T_1 = nk, \text{ where } \begin{array}{l} T = \text{Execution time} \\ n = \text{number of iterations} \\ \text{(instructions)} \\ k = \text{number of stages} \end{array}$$

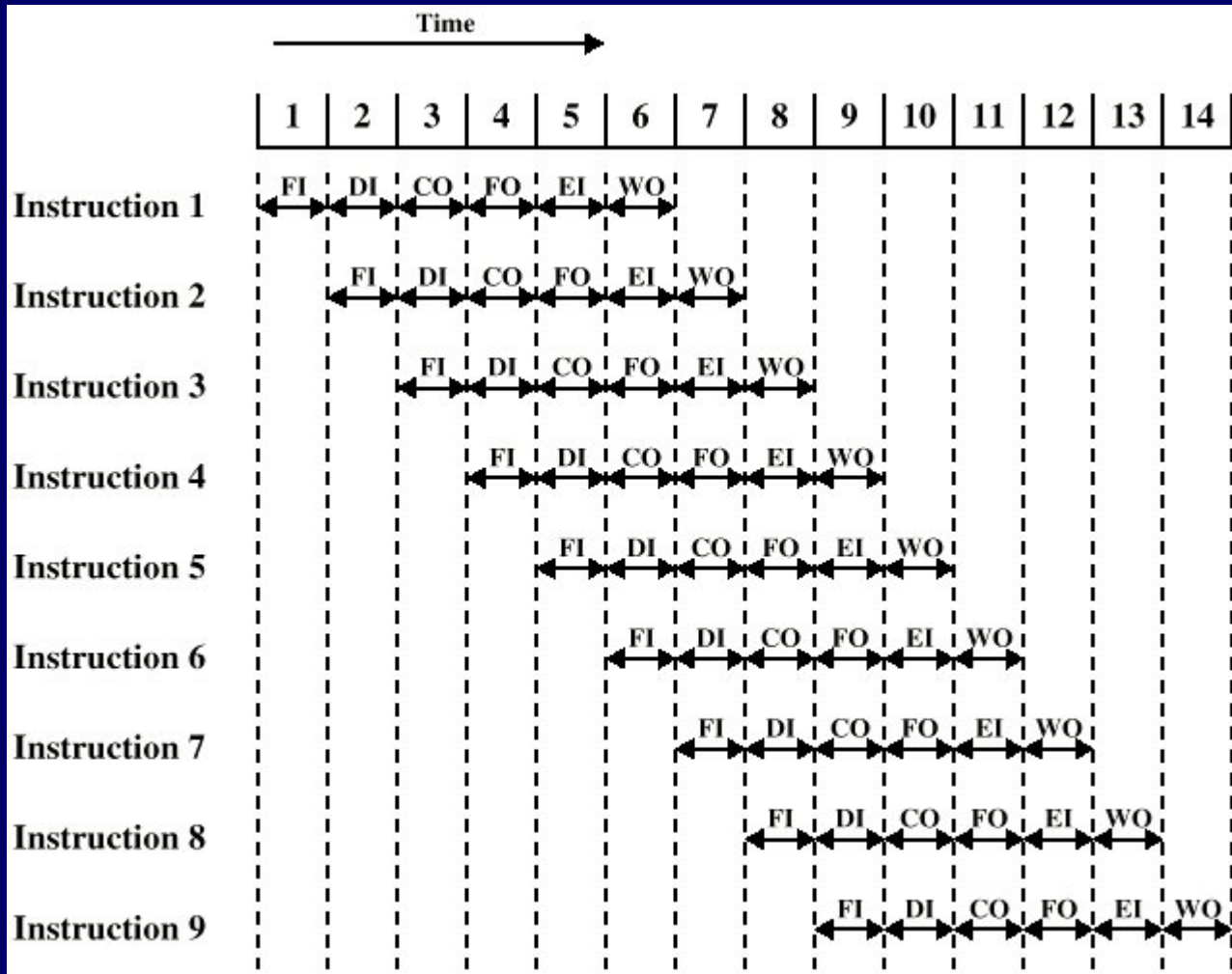
- With pipeline,

$$T_k = k + (n - 1)$$

- Speedup,

$$S = T_1 / T_k = nk / k + (n - 1)$$

Pipeline Performance (2)



Without pipeline:

$$T_1 = nk$$

$$= 9 \times 6$$

$$= 54$$

With pipeline:

$$T_k = k + (n-1)$$

$$= 6 + (9-1)$$

$$= 6 + 8$$

$$= 14$$

Speedup:

$$S = T_1 / T_k$$

$$= 54 / 14$$

$$= 3.86$$

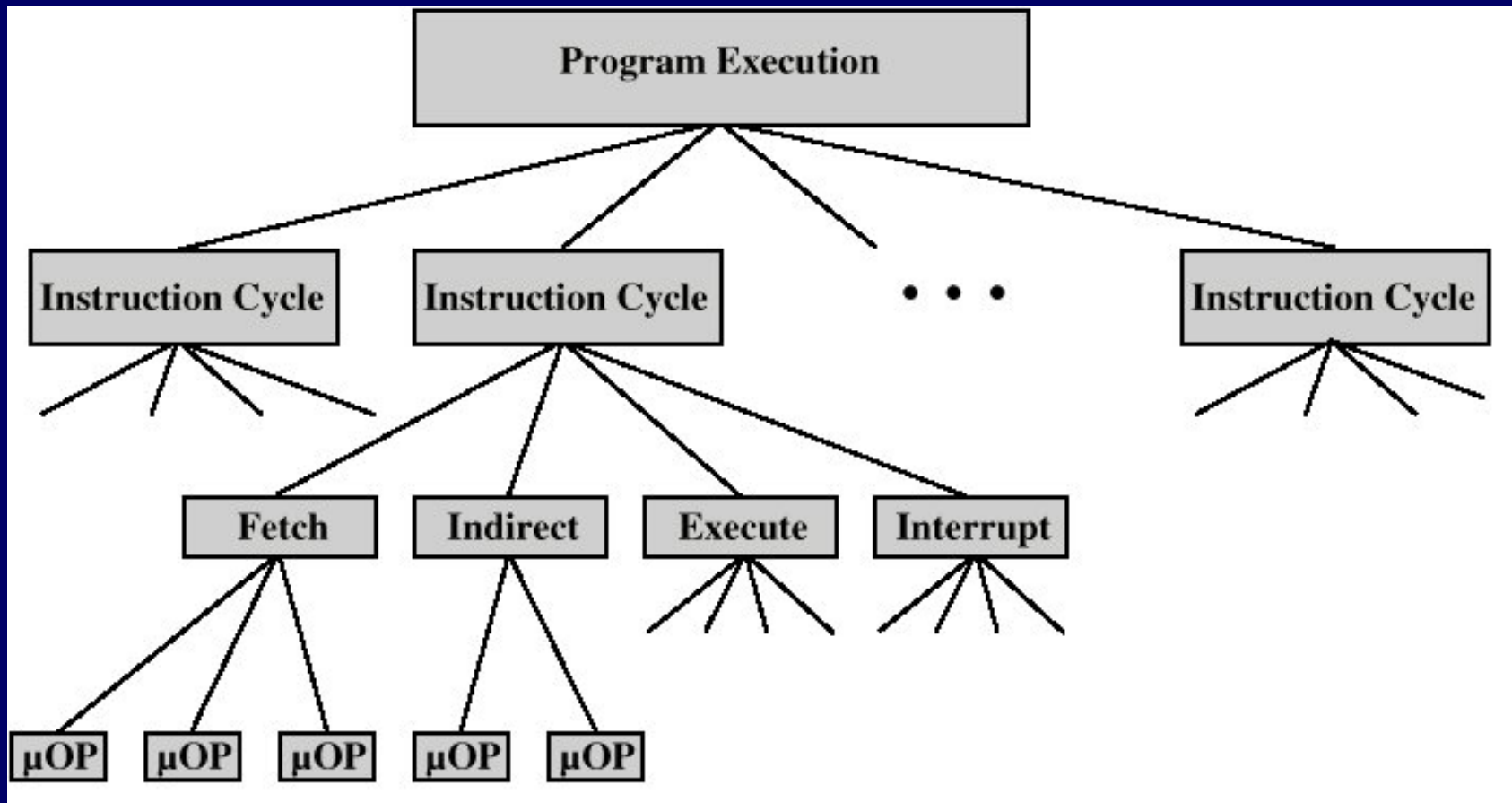
Computer Organisation

Topic 8 Control Unit

Micro-Operations

- A computer executes a program
- A program executes instructions sequentially (executed in an instruction cycle)
- Each instruction cycle made up of shorter subcycles
 - e.g. fetch, indirect, execute, interrupt
 - see pipelining
- Each subcycle involves shorter operations called micro-operations (each -op does very little, atomic operation of CPU)

Constituent Elements of Program Execution



Functions of Control Unit

- Control unit performs 2 basic tasks:
 - Sequencing
 - Execution
- Sequencing
 - Causing the CPU to step through a series of micro-operations in correct sequence
- Execution
 - Causing the performance of each micro-op
- This is done using Control Signals
 - i.e. control unit produces control signals

Control Signals - Inputs (1)

- For Control Unit to perform, it must have:
 - inputs to determine the state of the system
 - outputs to control the behaviour of the system
- Clock
 - One micro-instruction (or set of parallel micro-instructions) per clock cycle
- Instruction register
 - Op-code for current instruction
 - Determines which micro-instructions are performed

Control Signals - Inputs (2)

- Flags
 - To determine status of CPU
 - Results of previous ALU operations
 - e.g. ISZ instruction
- Control signals from control bus
 - Interrupts
 - Acknowledgements

Control Signals - Output

- Control signals within CPU (2 types)
 - Cause data movement (between registers)
 - Activate specific ALU functions
- Control signals to control bus (2 types)
 - To memory
 - To IO modules

Micro-programmed Control

- Use sequences of instructions (see earlier notes) to control complex operations
- In microprogramming language, each line describes set of micro-operations = microinstruction
- Sequence of microinstruction = microprogram
- Microprogramming a.k.a. firmware (because microprogram is like middleware between hardware and software)

Implementation (1)

- For each micro-op, all the control unit does is generate a set of control signals
- Each control signal (control line) is on or off
- This on/off state can be represented by a binary digit for each control line
- Have a control word for each micro-operation (each bit represents one control line)
- Have a sequence of control words for each machine code instruction (sequence of micro-ops)

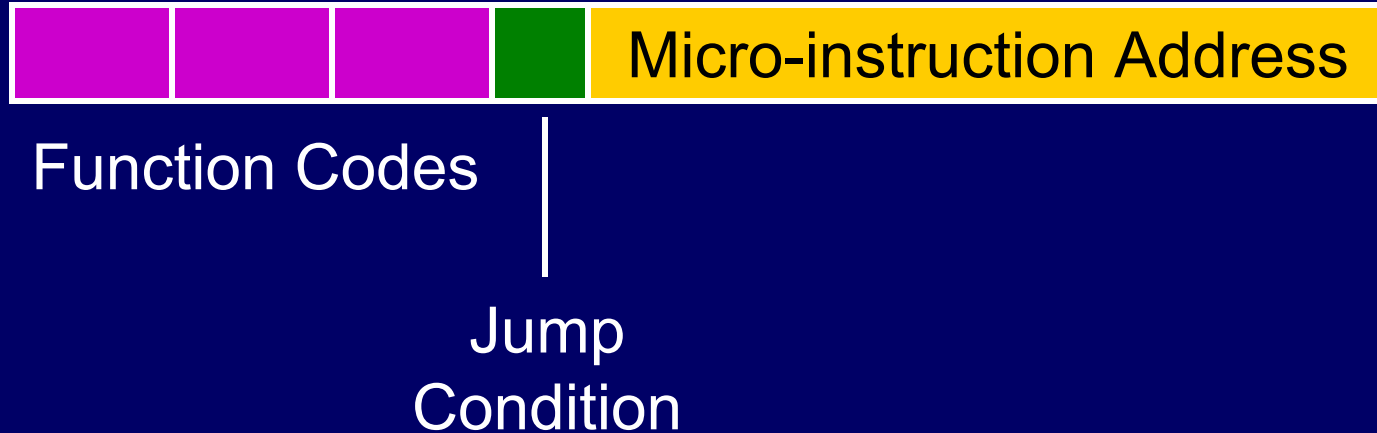
Implementation (2)

- Since the sequence of micro-ops are not fix, can add control words in memory, each has unique address
- Then, each control word can have address field to specify the next micro-instruction, depending on conditions
- Can also add few bits to indicate condition

Implementation (3)

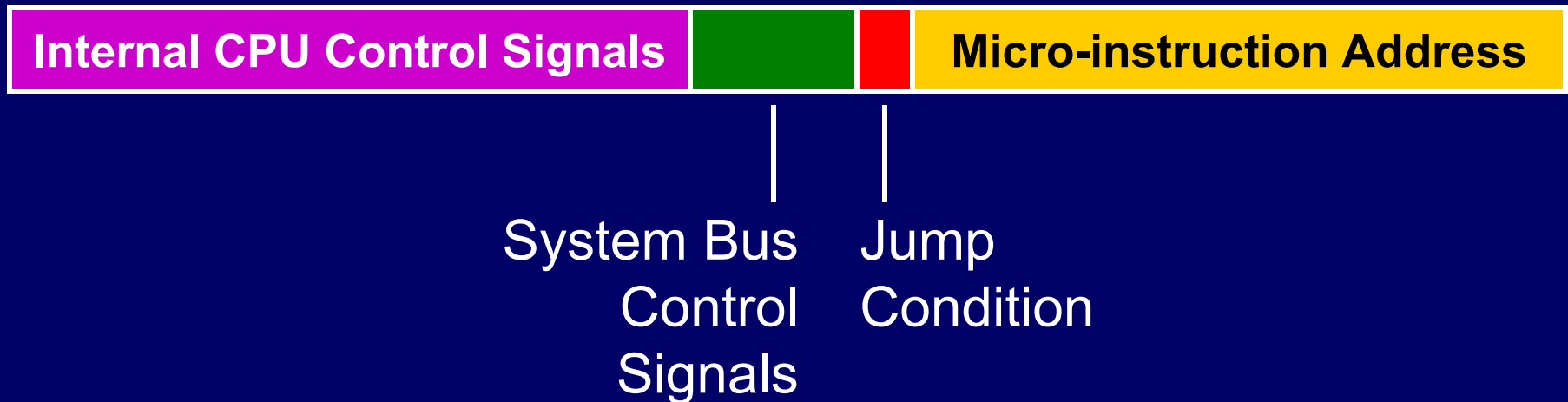
- Today's large microprocessor
 - Many instructions and associated register-level hardware
 - Many control points to be manipulated
- This results in control memory that
 - Contains a large number of words
 - co-responding to the number of instructions to be executed
 - Has a wide word width
 - Due to the large number of control points to be manipulated

Vertical Micro-programming (2)



Function codes are decoded to issue control signal

Horizontal Micro-programmed diag



Every bit in control field is attached to control line
(no decoding needed)

Control Unit Function (1)

- Control memory contains a program describing the behaviour of the control unit (a set of micro instructions)
 - Sequence logic unit issues READ command (executing the microinstruction)
 - Word specified in control address register (has address of next microinstruction) is read into control buffer register
 - Control buffer register contents generates control signals (to the control lines) and next address information (for the sequencing logic unit)
 - Sequence logic loads new address into control address register based on next address information from control buffer register and ALU flags
- 4 steps = 1 clock pulse

Like the PC

Control Unit Function (2)

- At end of each microinstruction, sequencing logic unit loads new address into control address register.
- Depending on value of ALU flags and control buffer register, one of 3 decisions is made:
 - Get next instruction: add 1 to control address register
 - Jump to new routine (jump microinstruction): load address field of control buffer register into control address register
 - Jump to machine instruction routine: load control address register based on opcode in IR

Next μ -
instruction

Next
instruction
cycle

Next
instruction

Computer Organisation

Topic 9 Interfacing and Communication

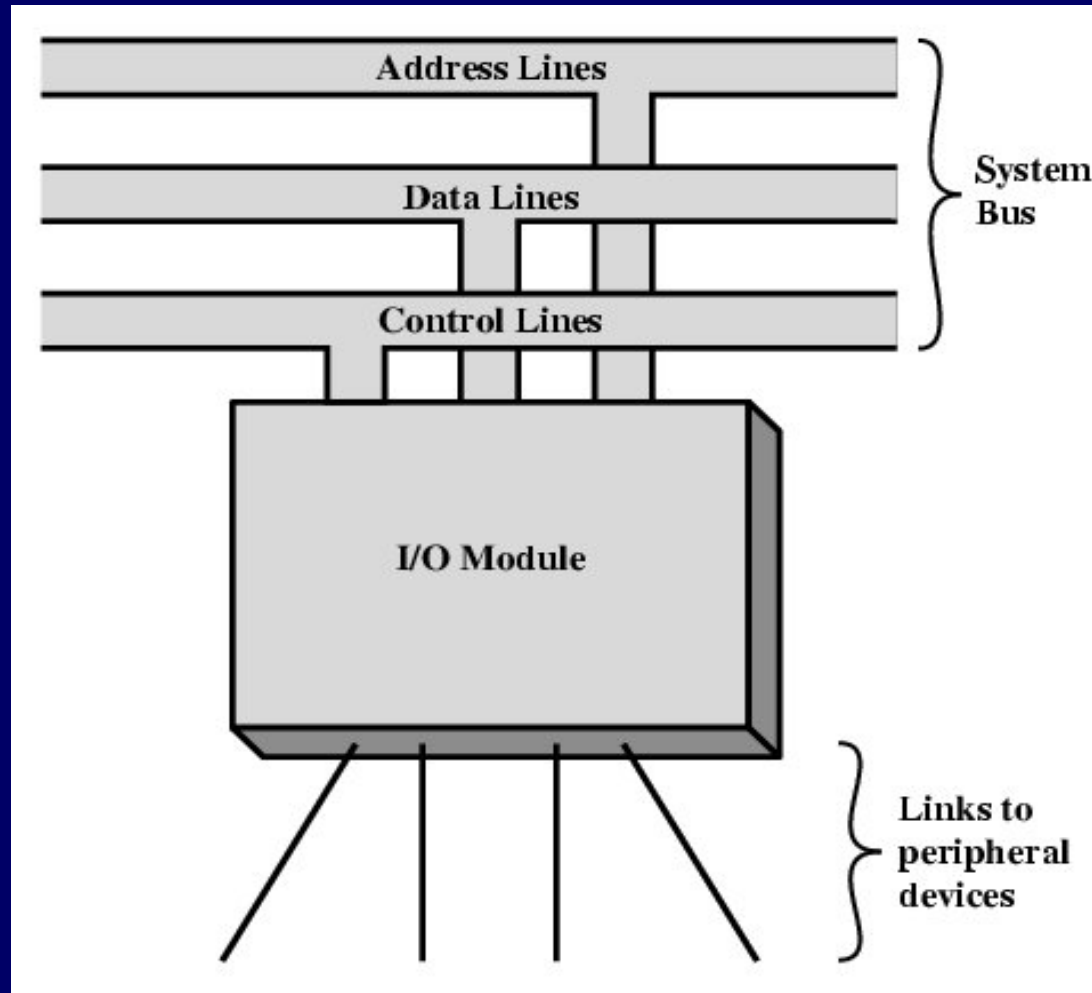
Input/Output Problems

- Why peripherals are not directly connected to system bus?
 - Wide variety of peripherals with various methods of operation
 - Delivering different amounts of data
 - At different speeds
 - In different formats and word lengths
- All slower than CPU and RAM
- Need I/O modules

Input/Output Module

- Interface to CPU and Memory via the system bus
- Interface to one or more peripherals by tailored data links
- Is designed
 - to provide a systematic means of controlling interaction between the devices and CPU/memory
 - To manage the I/O activity

Generic Model of I/O Module



I/O Module Decisions

- Hide the details of timing, formats or reveal device properties to CPU
- Support multiple or single device
- If I/O module controls most of device functions
 - I/O channel or I/O processor
 - In mainframe computers
- If I/O module is primitive
 - Leave much work for CPU
 - I/O controller or device controller

I/O Module

I/O Channel or
I/O Processor

I/O Controller or
Device Controller

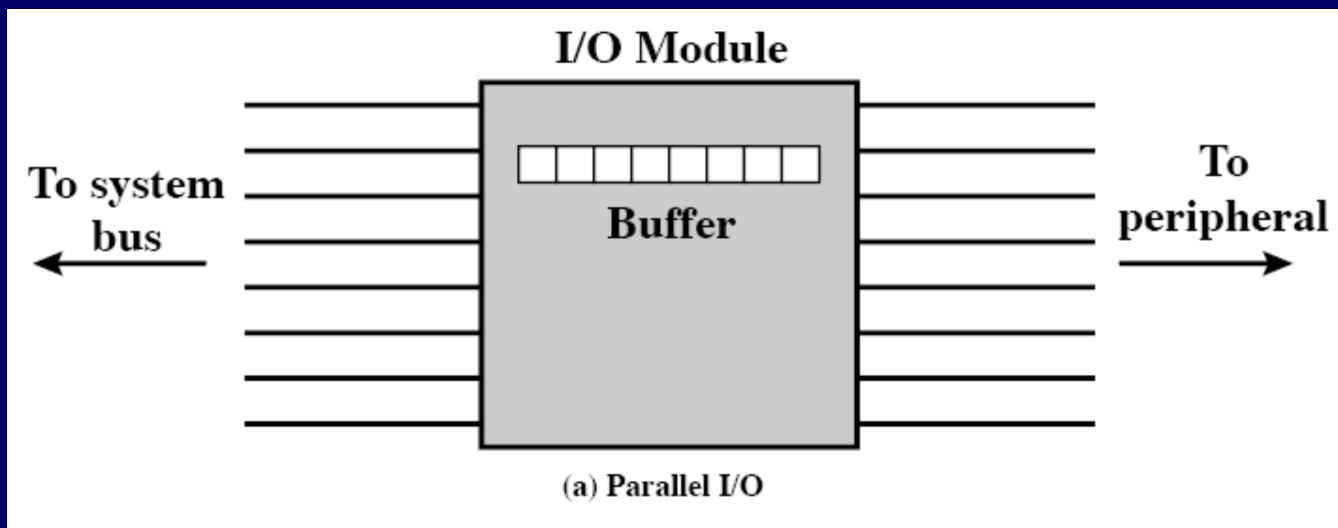
Input Output Operations/Techniques

- Programmed
- Interrupt driven
- Direct Memory Access (DMA)

	No Interrupt	Use Interrupt
I/O-to-memory transfer through processor	Programmed I/O	Interrupt-driven I/O
Direct I/O-to-memory transfer		Direct Memory Access (DMA)

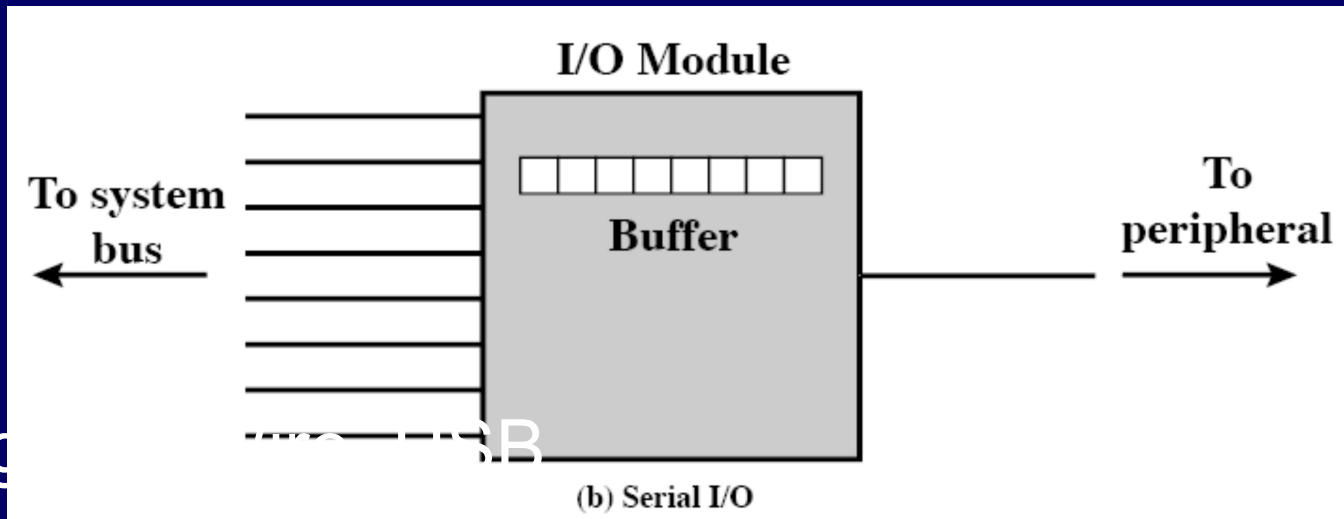
Interfacing

- Connecting devices together (from I/O module)
- Parallel interface
 - Multiple lines connecting I/O module and peripheral
 - Multiple bits transferred simultaneously
 - Higher speed peripherals, e.g. disks, printer



Interfacing

- Serial interface
 - Only one line used to transmit data
 - Bits transmitted one at a time
 - Slower peripherals, e.g. terminals, mice, joystick



- E.g. USB

Universal Serial Bus (USB)

- Simple and low cost
- Overcomes difficulties in limited number of I/O ports
- Wide range of I/O devices
- Plug-and-play

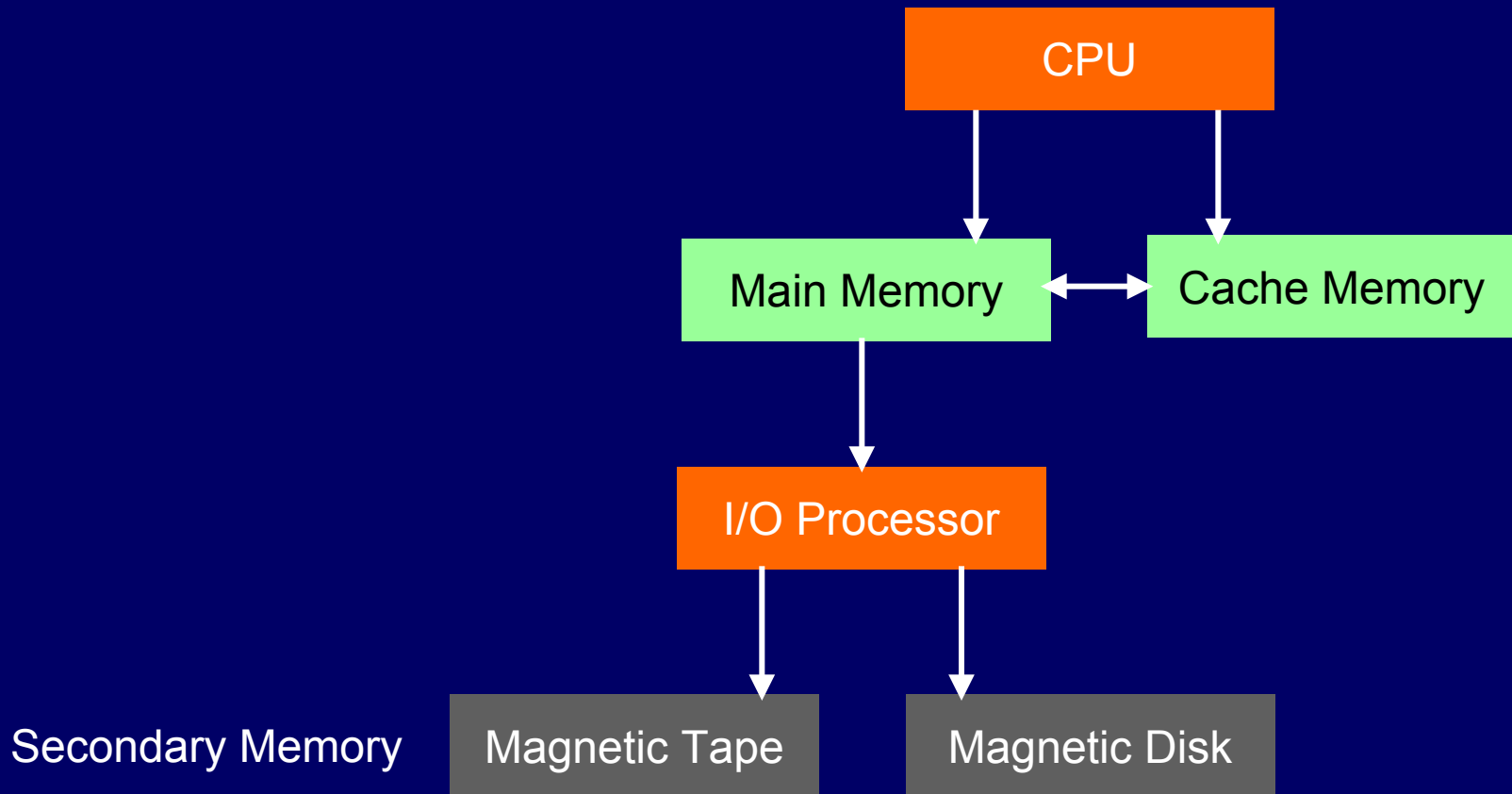
Computer Organisation

Topic 10 Memory

Memory

- For each classification there are several types of memories that can be differentiated by its making
- 2 classifications
 - Main Memory
 - Secondary Memory

Memory



Access Methods (1)

- Sequential
 - Memory is organized into unit of data, called records
 - Start at the beginning and read through in order (specific linear sequence)
 - Access time depends on location of data and previous location
 - e.g. tape
- Direct
 - Individual blocks have unique address
 - Access is by jumping to vicinity plus sequential search
 - Access time depends on location and previous location
 - e.g. disk

Access Methods (2)

- Random
 - Individual addresses identify locations exactly
 - Access time is independent of location or previous access
 - No “searching” required
 - e.g. RAM
- Associative
 - Data is located by a comparison with contents of a portion of the store
 - Word is retrieved based on a portion of content rather than on address
 - Access time is independent of location or previous access
 - e.g. cache

Main Memory

- Main memory implementation using integrated circuit
- Main memory stores program and data
- Computer speed = speed of data transfer from main memory
- Physical size of main memory is usually not as big as the specified address by program
 - Only blocks that are needed are stored in main memory
 - Others - secondary memory
- Maximum size of main memory depends on the addressing scheme

Random Access Memory (RAM)

- Memory unit that can be accessed for read and write operation
- 2 types of RAM
 - Static RAM (SRAM)
 - Dynamic RAM (DRAM)

Error Correction

- Hard Failure
 - Permanent physical defect
 - Caused by harsh environmental abuse, wear, etc.
- Soft Error
 - Random, non-destructive
 - No permanent damage to memory
 - Caused by power supply problems, alpha particles, etc.
- Memory needs logic for detecting and correcting errors

Error Correcting Code Function

- When data written into memory
 - Calculation function, f , is performed and produces a code
 - Both data (M bits) and code (K bits) are stored in memory
- When data read out
 - Data (M bits) used to generate a new code (K bits) using same function, f
 - Both codes (before memory write and after memory read) are compared

Error Correcting Code Function

- Three results
 - No errors
 - Error detected
 - Possible to correct by feeding data bits and error correction bits into a corrector
 - Error detected
 - Not possible to correct
 - Report error
- Codes = error correcting codes
- Simplest error correcting codes
 - Hamming error correcting code

Computer Organisation

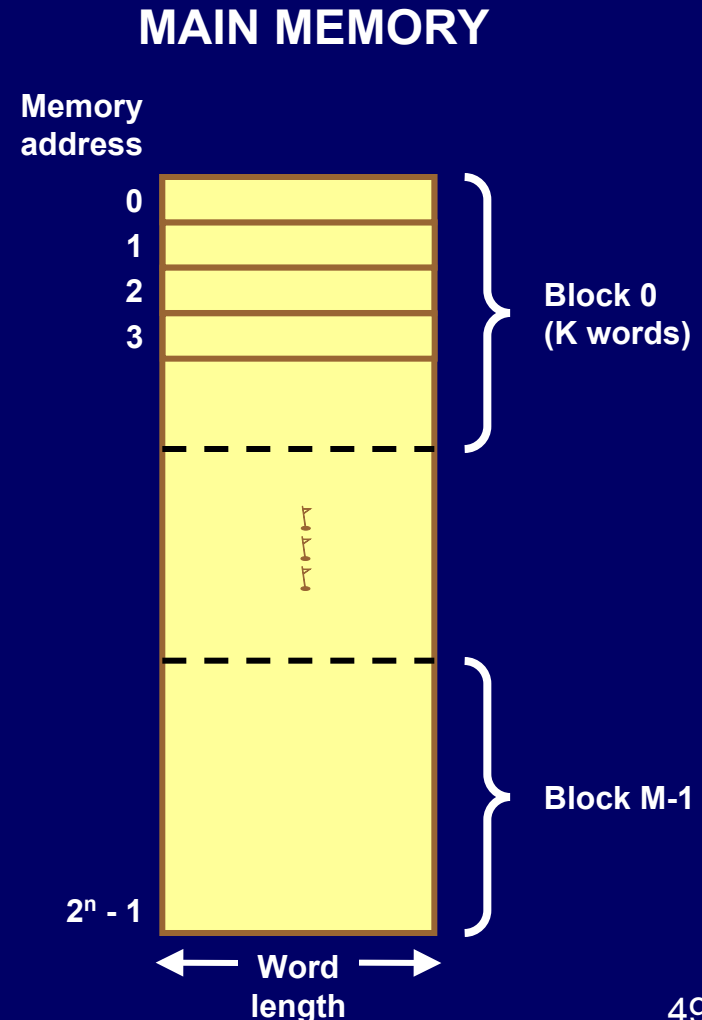
Topic 11 Cache Memory

Cache Operation - Overview

- CPU requests contents of memory location
- Check cache for this data
 - If present, get from cache (fast)
 - If not present, read required block from main memory to cache
 - Then deliver from cache to CPU
- Locality of reference
 - When a block of data is fetched into cache to satisfy a single memory reference, it is likely there will be future reference to that same location

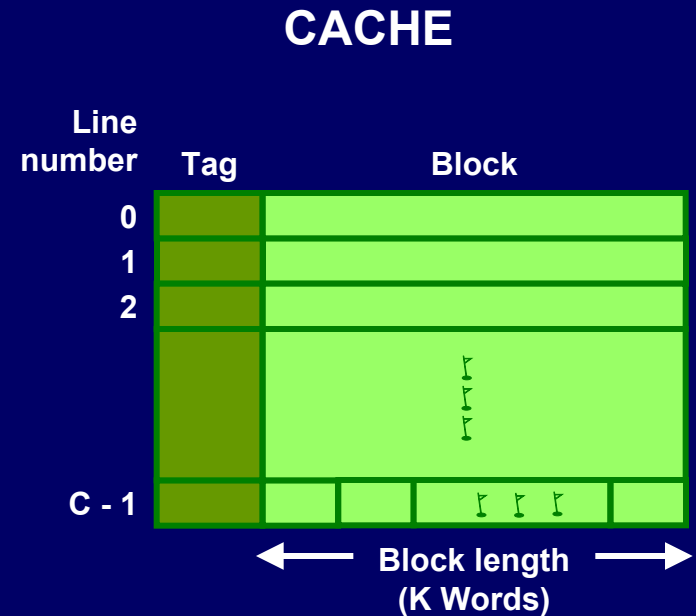
Cache/Main Memory Structure

- Main memory: 2^n addressable words
 - Each word has unique n bit address
- Main memory considered to consist of M number of blocks
 - Each block has K words
 - $M = 2^n / K$ blocks

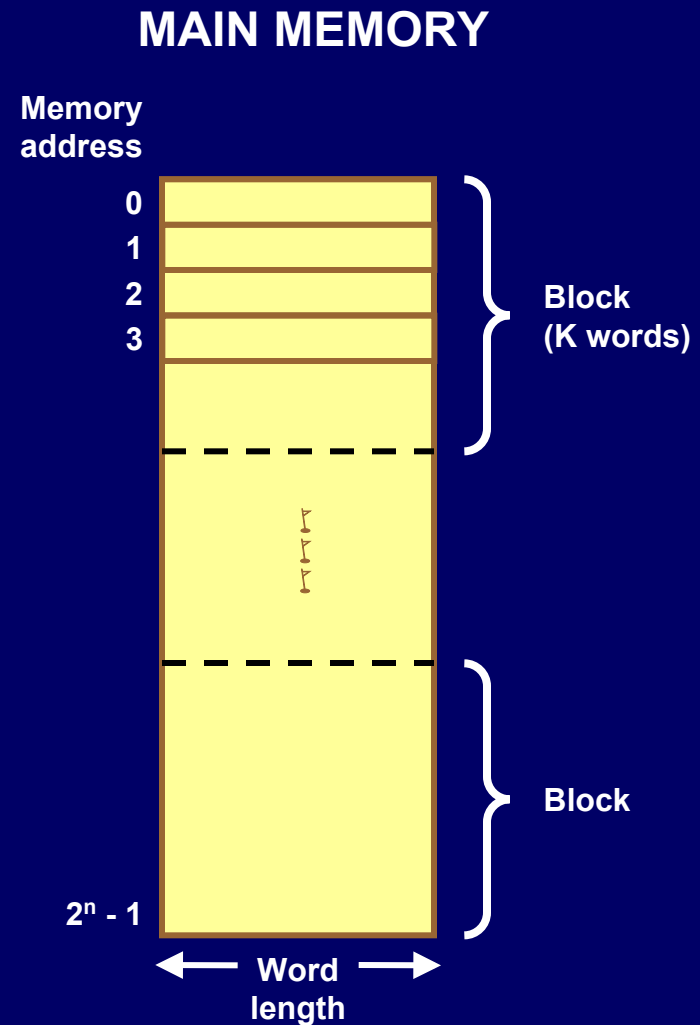
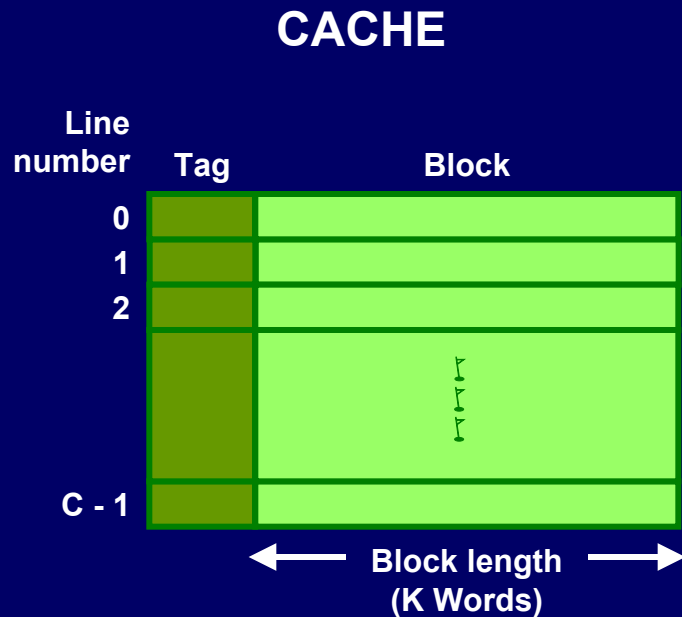


Cache/Main Memory Structure

- Cache consists of C lines of K words each
 - Number of lines is much less than number of main memory blocks ($C \ll M$)
- Since there are more blocks than lines, an individual line cannot be permanently dedicated to a particular block
 - Cache includes tags to identify which block of main memory is in each line



Cache/Main Memory Structure



Mapping Function

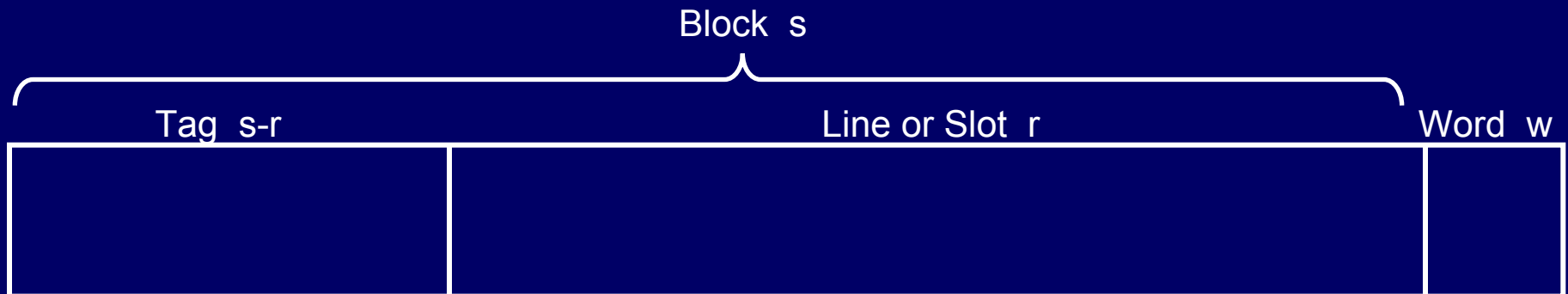
- Fewer cache lines than main memory blocks
 - Algorithm needed for mapping main memory blocks into cache lines
- Need to determine which main memory block currently occupies a cache line
- Three mapping techniques
 - Direct
 - Associative
 - Set associative

Direct Mapping

- Each block of main memory maps to only one cache line
 - i.e. if a block is in cache, it must be in one specific place
- Mapping (to a line number) expressed as
 - $i = j \text{ modulo } m$
 - i = cache line number
 - j = main memory block number
 - m = number of lines in cache

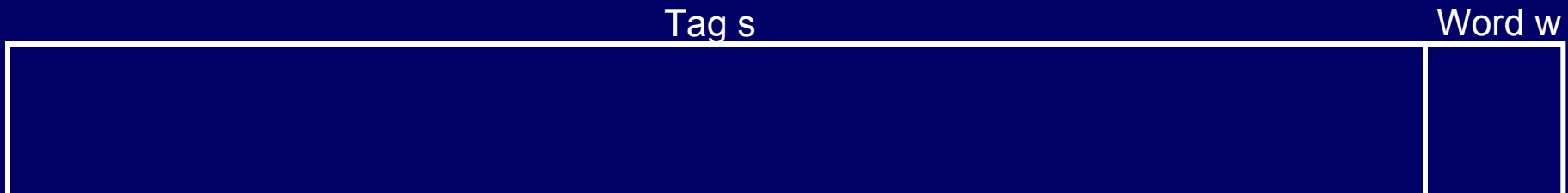
Direct Mapping Summary

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $2^{s+w}/2^w = 2^s$
- Number of lines in cache = $m = 2^r$
- Size of tag = $(s - r)$ bits



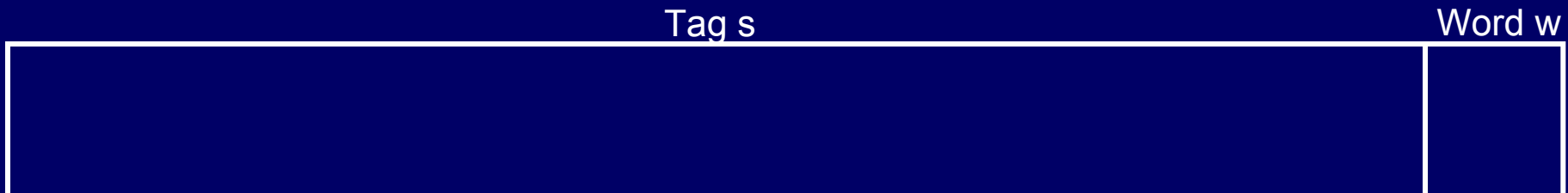
Associative Mapping

- A main memory block can load into any line of cache
- Memory address is interpreted as tag and word
- Tag uniquely identifies block of memory
- Every line's tag is examined for a match
- Cache searching gets expensive



Associative Mapping Summary

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $2^{s+w}/2^w = 2^s$
- Number of lines in cache = undetermined
- Size of tag = s bits

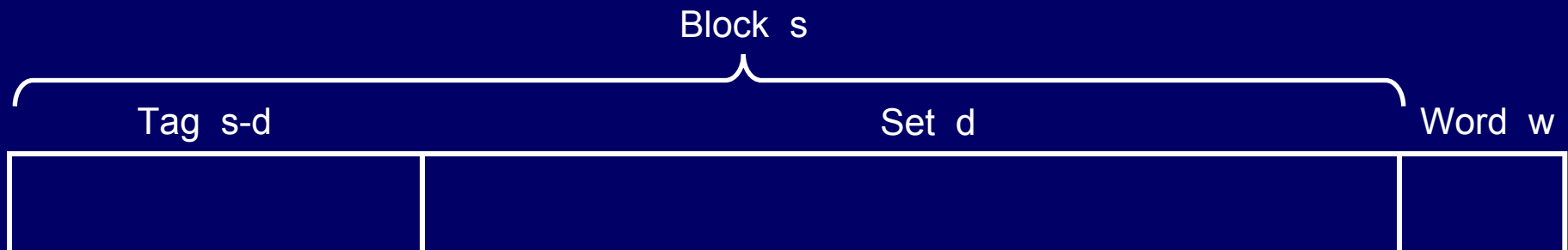


Set Associative Mapping

- Cache is divided into a number of sets
- Each set contains a number of lines
- A given block maps to any line in a given set
 - e.g. Block B can be in any line of set i
- e.g. 2 lines per set
 - 2 way associative mapping
 - A given block can be in one of 2 lines in only one set
- Mapping (to a set number) expressed as
 - $i = j \text{ modulo } v$
 - i = cache set number
 - j = main memory block number
 - v = number of sets in cache

Set Associative Mapping Summary

- Address length = $(s + w)$ bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = 2^s
- Number of lines in set = k
- Number of sets = $v = 2^d$
- Number of lines in cache = $kv = k * 2^d$
- Size of tag = $(s - d)$ bits



Computer Organisation

Topic 12

Memory Management and Virtual Memory

Memory Management

- Uni-program
 - Memory split into two
 - One for Operating System (monitor)
 - One for currently executing program
- Multi-program
 - “User” part is sub-divided and shared among active processes

Swapping

- Problem: I/O is so slow compared with CPU that even in multi-programming system, CPU can be idle most of the time
- Solutions:
 - Increase main memory
 - Expensive
 - Leads to larger programs
 - Swapping

What is Swapping?

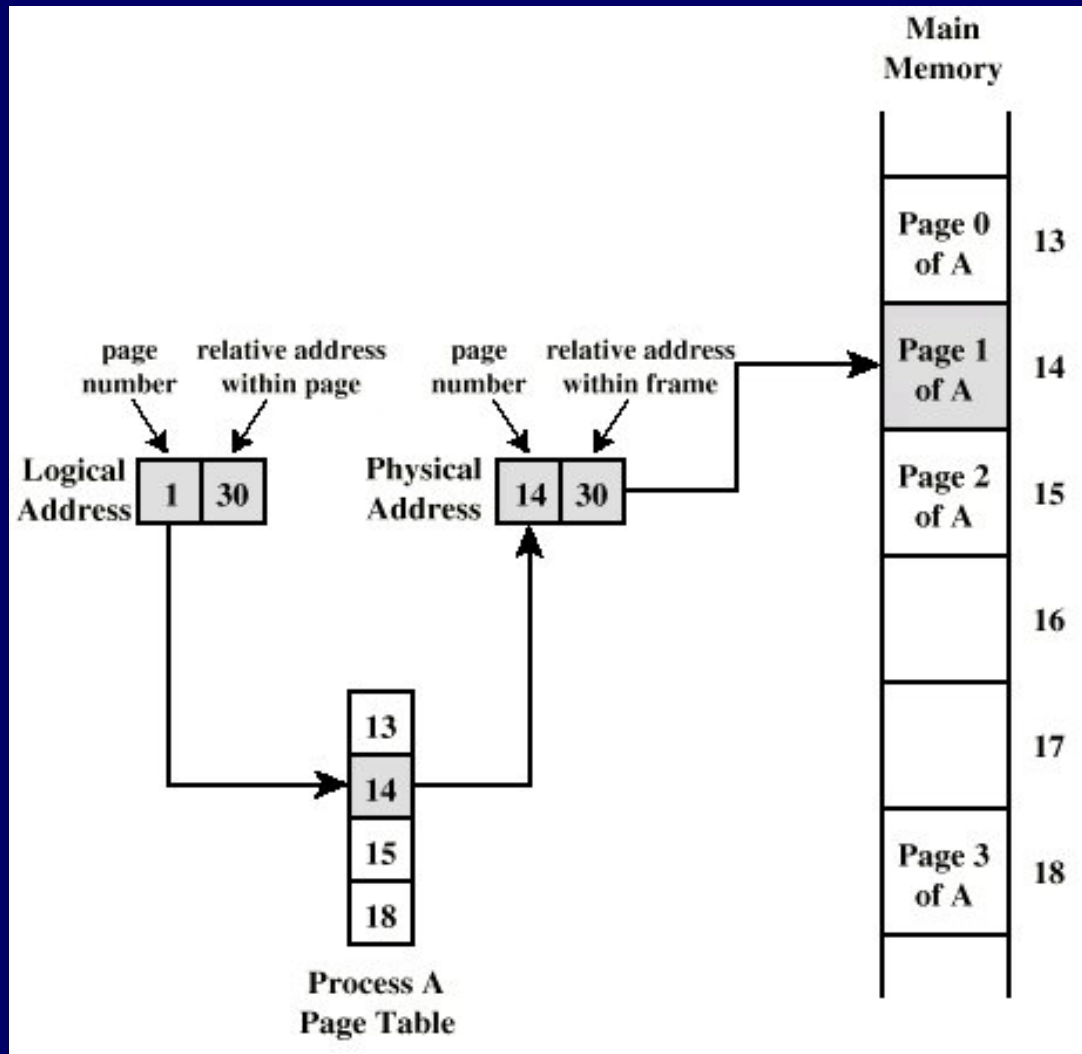
- Long term queue of processes stored on disk
- Processes “swapped” in as space becomes available
- As a process completes it is moved out of main memory
- If none of the processes in memory are ready (i.e. all I/O blocked)
 - Swap out a blocked process to intermediate queue
 - Swap in a ready process or a new process
 - But swapping is an I/O process...

Paging

- Split memory into equal sized, small chunks: page frames
- Split programs (processes) into equal sized small chunks: pages
- Allocate the required number page frames to a process
- Operating System maintains list of free frames
- A process does not require contiguous page frames
- Use page table to keep track

Paging

Logical and Physical Addresses



Virtual Memory

- Demand paging
 - Do not require all pages of a process in memory
 - Bring in pages as required
 - If page not in main memory (branch or requires data), page fault is triggered
- Page fault
 - Required page is not in memory
 - Operating System must swap in required page
 - May need to swap out a page to make space
 - Select page to throw out based on recent history
 - If page thrown out just before it is used, it must be immediately brought in again
 - Too many of this can lead to “trashing”

Thrashing

- Too many processes in too little memory
- Operating System spends all its time swapping
- Little or no real work is done
- Disk light is on all the time
- Solutions
 - Good page replacement algorithms
 - Reduce number of processes running
 - Fit more memory

Computer Organisation

Topic 13 Multiprocessor Organisation

Multiple Processor Organization

- Flynn's Classification
 - Single instruction, single data stream - SISD
 - Single instruction, multiple data stream - SIMD
 - Multiple instruction, single data stream - MISD
 - Multiple instruction, multiple data stream - MIMD

Single Instruction, Single Data Stream - SISD

- Single processor
- Single instruction stream
- Data stored in single memory
- Uni-processor

Single Instruction, Multiple Data Stream - SIMD

- Single machine instruction
- Controls simultaneous execution
- Number of processing elements
- Lockstep basis
- Each processing element has associated data memory
- Each instruction executed on different set of data by different processors
- Vector and array processors

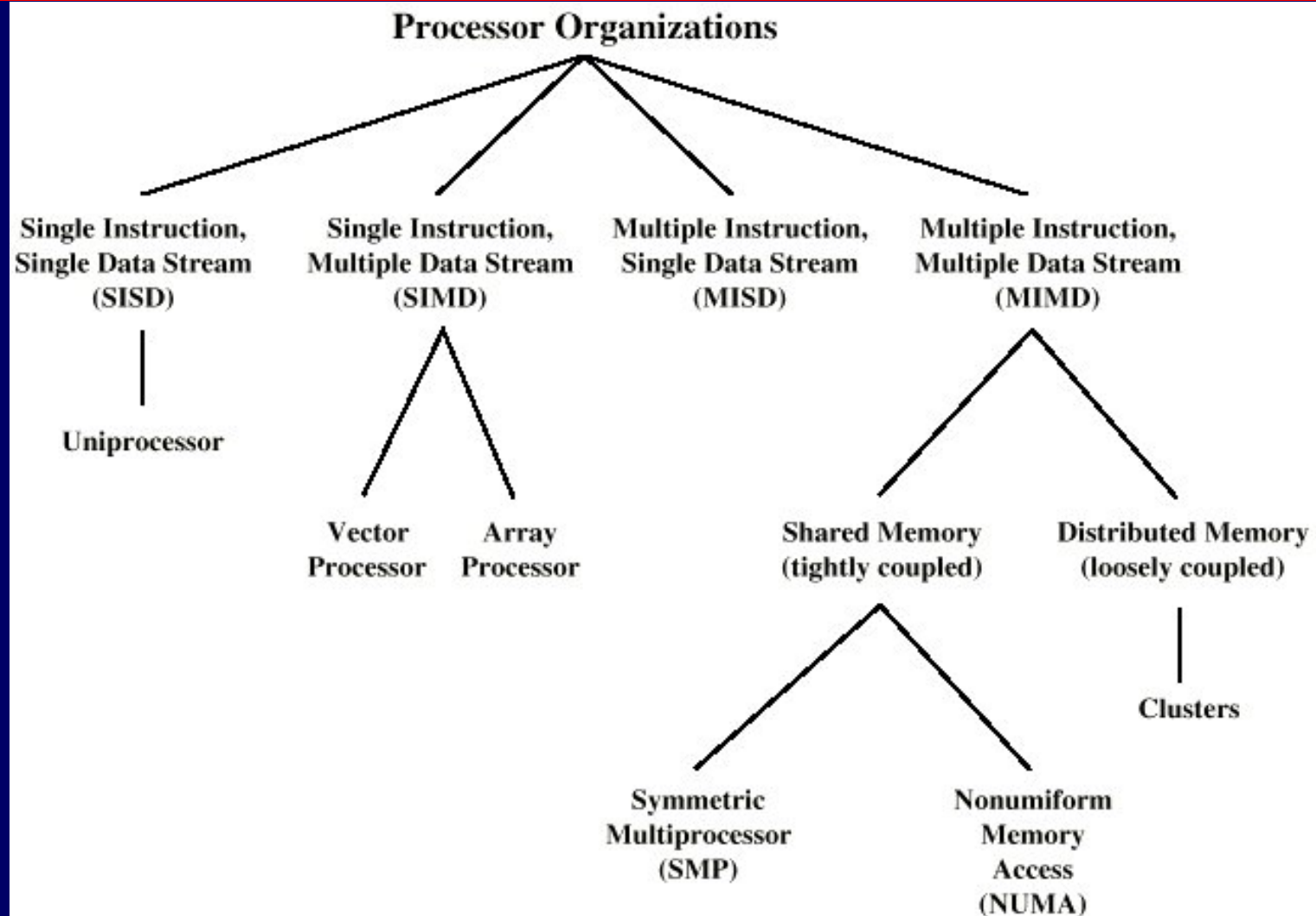
Multiple Instruction, Single Data Stream - MISD

- Sequence of data
- Transmitted to set of processors
- Each processor executes different instruction sequence
- Never been implemented

Multiple Instruction, Multiple Data Stream- MIMD

- Set of processors
- Simultaneously execute different instruction sequences
- Different sets of data
- SMPs, clusters and NUMA systems

Taxonomy of Parallel Processor Architectures



Symmetric Multiprocessors (1)

- A stand alone computer with the following characteristics
 - Two or more similar processors of comparable capacity
 - Processors share same memory and I/O
 - Processors are connected by a bus or other internal connection
 - Memory access time is approximately the same for each processor
 - All processors share access to I/O
 - Either through same channels or different channels giving paths to same devices

Symmetric Multiprocessors (2)

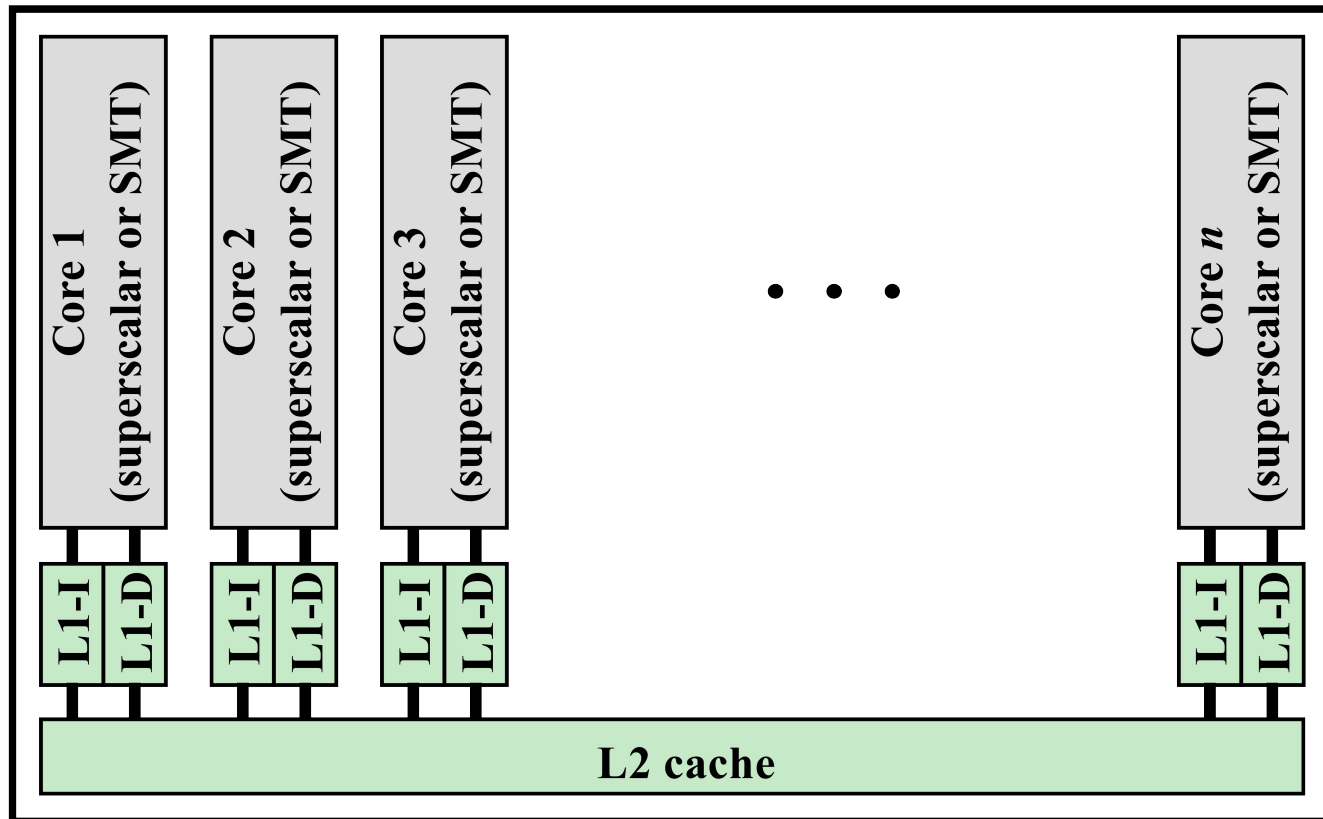
- A stand alone computer with the following characteristics (cont)
 - All processors can perform the same functions (hence symmetric)
 - System controlled by integrated operating system
 - providing interaction between processors
 - Interaction at job, task, file and data element levels

Multicore Computer

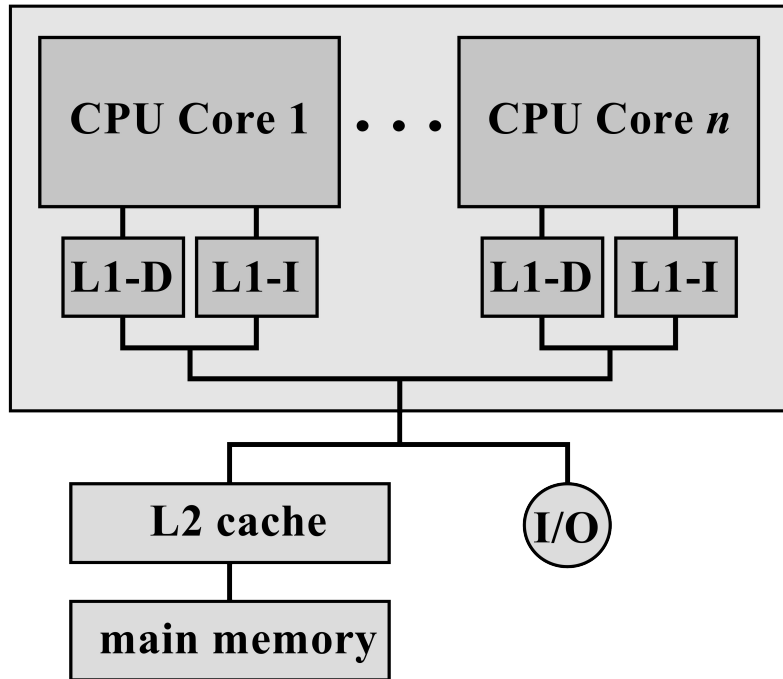
- Core
 - “Brain” of the CPU
 - Receives instructions, performs calculations or operations, etc.
- A CPU may have multiple cores.
 - 2 cores = dual-core
 - 4 cores = quad-core
 - etc.

Alternative Chip Organizations

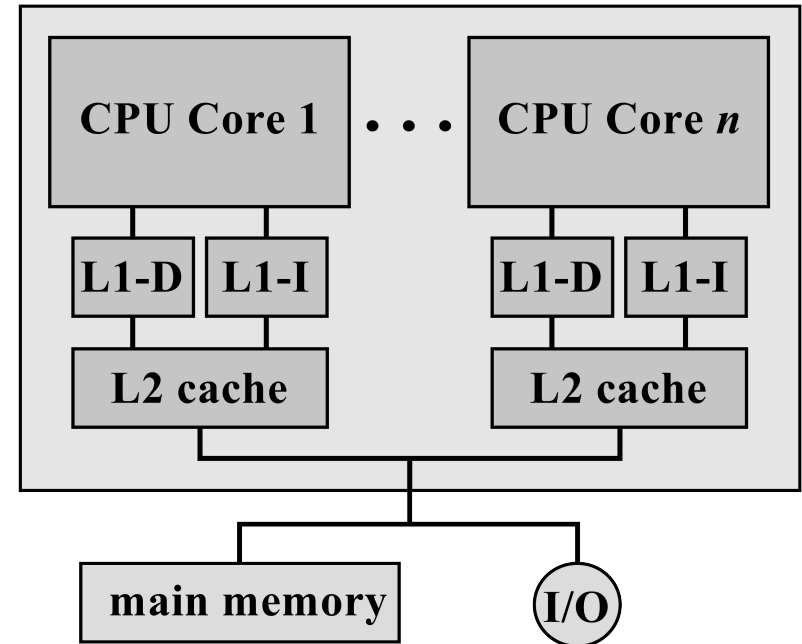
- Multicore



Multicore Organization Alternatives



(a) Dedicated L1 cache



(b) Dedicated L2 cache